

Collecting Data from the Web

From Query to Corpus: The Practical Workflow

Moses Boudourides

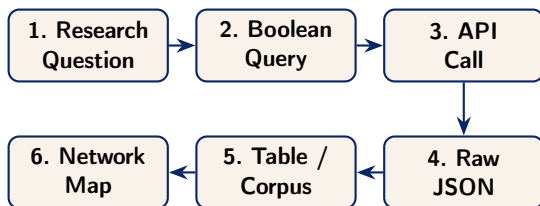
*Faculty, Graduate Program on Data Science
Northwestern University*

Moses.Boudourides@northwestern.edu

instats Seminar

Wednesday, June 10, 2026

The Day 1 Workflow at a Glance



- Slides 1 explained **what** APIs and JSON are (the plumbing).
- This deck explains **how** to use them: turning a research question into a downloadable corpus.
- Each step in the chain above is a decision that shapes the final dataset.

From Research Question to Boolean Query

A Boolean query translates a conceptual question into a string the API can search.

Example

Research question: Does social media use affect adolescent mental health?

Boolean query:

"social media" AND ("mental health" OR depression OR anxiety) AND adolescen*

Three operators you must know:

- AND — both terms must appear (narrows results).
- OR — either term may appear (broadens results).
- * — wildcard: adolescen* matches *adolescent*, *adolescents*, *adolescence*.

Query Design: Common Mistakes

Too Broad

health AND technology

Returns hundreds of thousands of papers across completely unrelated fields.

Fix: Add a third concept or a field filter.

Too Narrow

"wearable ECG monitor" AND "atrial fibrillation detection" AND "elderly patients"

Returns zero or five papers.

Fix: Replace one phrase with an OR group of synonyms.

The Goldilocks Rule

Aim for **200–2,000 results**. The keyword network will tell you whether the query is on target.

The Scholarly APIs: Which One Should You Use?

	OpenAlex	Crossref	Europe PMC
Coverage	All disciplines	All disciplines	Life sciences & biomedicine
Records	~250M works	~160M DOIs	~42M abstracts
Abstracts	~87%	Partial	Near-complete
Full text	No	No	Yes (OA articles)
Key strength	Concepts & networks	Publisher meta-data	Full-text search
Rate limit	100k/day (polite pool)	Polite pool	2.5M/day

Rule of thumb: Start with OpenAlex. Switch to Europe PMC for biomedical topics where full-text indexing matters.

API Keys and Authentication

Some APIs require you to identify yourself before they will respond.

No Key Required

- OpenAlex (anonymous or polite pool)
- Crossref (anonymous or polite pool)
- Europe PMC
- World Bank

Just add your email as a courtesy:

`&mailto=you@uni.edu`

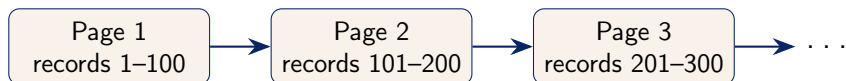
Key Required

- Scopus / Web of Science
- NIH RePORTER (POST)
- Zotero (OAuth token)

Never share your key in public code. Use environment variables or a `.env` file.

Pagination: Getting More Than One Page of Results

APIs never return all results at once. They split them into **pages**.



Two pagination styles used by the APIs in this seminar:

- **Offset pagination** (Crossref, Europe PMC):
`&offset=100&limit=100`. Simple but slow for large corpora.
- **Cursor pagination** (OpenAlex): the API returns a `next_cursor` token; pass it back on the next request. Faster and more reliable.

Rate Limits: Being a Polite API User

APIs enforce **rate limits** to prevent overloading their servers.

What happens if you exceed the limit?

The server returns a **429 Too Many Requests** status code and stops responding — sometimes for hours.

Best practices:

- Add a short **sleep** between requests (`time.sleep(1)` in Python).
- Use the **polite pool**: add `&mailto=you@email.com` to OpenAlex and Crossref requests. This gives you a higher rate limit and faster responses.
- Request the maximum allowed records per page (e.g., `per_page=200`) to reduce the total number of requests.
- **Cache** results locally — never re-download data you already have.

OpenAlex in Depth: Key Fields in the Response

When OpenAlex returns a record, the most useful fields for a systematic review are:

Field	What it contains
title	Full title of the work
publication_year	Year of publication
doi	Digital Object Identifier (unique ID)
authorships	List of authors with ORCID and institution
concepts	AI-assigned topic tags with relevance scores
cited_by_count	Number of times this work has been cited
open_access.oa_url	Link to free full text (if available)
abstract_inverted_index	Abstract stored as a word-position map

Note: The `abstract_inverted_index` must be reconstructed into readable text — the app does this automatically.

The Inverted Index: Why Abstracts Look Odd in Raw JSON

OpenAlex does not store abstracts as plain text. It stores them as an **inverted index** — a dictionary mapping each word to its position(s).

Raw JSON (what the API returns):

```
{  
  "This": [0],  
  "study": [1],  
  "examines": [2],  
  "climate": [3],  
  "change": [4]  
}
```

Reconstructed abstract:

"This study examines climate change ..."

Why? Storing word positions instead of full text reduces storage by ~60% and speeds up full-text search. The app reconstructs the text automatically.

Deduplication: Why the Same Paper Appears Twice

When you query multiple APIs or run multiple queries, the same paper often appears more than once.

Common sources of duplicates:

- The same paper is indexed by both OpenAlex and Crossref.
- A preprint and its published version are both returned.
- Slightly different DOI formats (10.1000/xyz vs <https://doi.org/10.1000/xyz>).
- Title-based duplicates when no DOI is available.

Deduplication strategy used in the app:

- 1 Match on DOI (exact, case-insensitive, after stripping the <https://doi.org/> prefix).
- 2 If no DOI, match on normalised title (lowercase, punctuation removed).
- 3 Keep the record with the most complete metadata.

What is a Keyword Co-occurrence Network?

The app produces a **keyword co-occurrence network** from the retrieved corpus. What does this mean?

- Each **node** is a keyword (concept tag) that appears in at least one paper.
- Two nodes are connected by an **edge** if they appear together in the same paper.
- The **size** of a node reflects how often that keyword appears across the corpus.
- The **colour** of a node reflects its community (cluster of closely related concepts).

What to look for

- **Central nodes:** core concepts of the field.
- **Peripheral nodes:** niche or emerging sub-topics.
- **Bridges:** nodes connecting two clusters — often the most interesting interdisciplinary papers.

Reading the Keyword Network: A Worked Example

Query: machine learning clinical diagnosis prediction

What a well-formed network looks like:

- Large central nodes: *machine learning, deep learning, clinical decision support*.
- Two or three distinct clusters: imaging, NLP, tabular data.
- A bridge node: *electronic health records* — connecting the imaging and NLP clusters.

Warning signs in the network:

- One giant cluster with no sub-structure $\Rightarrow$ query is too broad.
- Dozens of isolated nodes with no edges $\Rightarrow$ query is too narrow.
- The central node is a generic term like *study* or *analysis* $\Rightarrow$ add domain-specific terms.

The Publication Year Distribution: What It Tells You

The app plots the number of papers per year retrieved by your query. This chart is more informative than it looks.

Pattern	Interpretation
Exponential growth in the last 5 years	Rapidly emerging field; literature may not yet be consolidated.
Flat or declining trend	Mature or declining field; foundational papers may dominate.
Sharp spike in one year	Likely triggered by a landmark paper, policy change, or crisis event.
Almost no papers before 2015	The concept itself is new — check if earlier synonyms exist.

Practical tip: If your query returns very few papers before 2010, consider adding older synonyms (e.g., “artificial intelligence” instead of “large language models”) to capture the historical literature.

Exporting the Corpus: File Formats

The app offers three export formats. Each serves a different downstream purpose.

Format	Best used for	Opens in
CSV	Quantitative analysis, data cleaning (Day 2), Python/R scripts	Excel, Python, R
RIS	Importing into reference managers (Zotero, Mendeley, End-Note) for Day 2 screening	Zotero, Mendeley
Parquet	Large corpora (>50k records); compressed and fast to reload	Python (pandas), R (arrow)

Recommendation for this seminar: Download **CSV** for Day 2 data cleaning and **RIS** if you plan to use Zotero for screening.

Translating Your Research Question: PICO and SPIDER

Structured frameworks help translate a research question into a Boolean query.

PICO (Health Sciences)

- **Population:** Who? (e.g., *elderly patients*)
- **Intervention:** What? (e.g., *telehealth*)
- **Comparison:** Compared to? (e.g., *in-person care*)
- **Outcome:** Measured how? (e.g., *readmission rates*)

SPIDER (Social Sciences)

- **Sample:** Who?
- **Phenomenon of Interest:** What?
- **Design:** What study type?
- **Evaluation:** What outcome?
- **Research type:** Qual or quant?

Filters: Narrowing the Corpus Without Changing the Query

All three APIs support **filters** that restrict results without adding terms to the Boolean string.

Filter	API parameter	Example
Publication year	<code>filter=publication_year:2015-2024</code>	Restrict to the last decade
Document type	<code>filter=type:article</code>	Exclude books, datasets, reviews
Open Access	<code>filter=is_oa:true</code>	Only papers with free full text
Language	<code>filter=language:en</code>	English-language papers only
Journal / Source	<code>filter=primary_location.source.id:S...</code>	Restrict to a specific journal

Filters are applied server-side before pagination — they reduce the total result count, not just what you see on the first page.

Research Ethics and Terms of Service

Programmatic data collection comes with ethical and legal obligations.

Always check before you collect

- **Terms of Service:** Does the API's ToS permit bulk download and research use? (OpenAlex: yes. Scopus: institutional licence required.)
- **robots.txt:** For web scraping, check `/robots.txt` on the target site. Respect Disallow directives.
- **Personal data:** Author names and affiliations are public, but email addresses scraped from PDFs may be subject to GDPR.
- **Attribution:** Cite the data source in your methods section. OpenAlex requests citation of Priem et al. (2022).

Reproducibility: Logging Your Query

A corpus constructed via API is only reproducible if the query is fully documented.

What to record in your methods section:

- 1 The **API** used (name, version, base URL).
- 2 The exact **Boolean query string**.
- 3 All **filters** applied (year range, document type, language).
- 4 The **date** the query was run (results change as the index is updated).
- 5 The **total number of records** returned before and after deduplication.

PRISMA-S requirement

The PRISMA-S extension (Rethlefsen et al., 2021) requires that all of the above be reported for any API-based literature search submitted for peer review.

Summary: The Day 1 Decision Checklist

Before you run your first query, work through this checklist:

✓ Decision

- ☐ I have a clear research question (PICO / SPIDER).
 - ☐ I have translated it into a Boolean query with AND / OR / wildcards.
 - ☐ I have chosen the right API for my discipline.
 - ☐ I have set appropriate filters (year, type, language).
 - ☐ I have checked the API's terms of service.
 - ☐ I have added my email to the polite pool.
 - ☐ I have logged the query string, filters, and date.
 - ☐ I have inspected the keyword network to validate the query.
 - ☐ I have exported the corpus in the format I need for Day 2.
-